



Budapesti Műszaki és Gazdaságtudományi Egyetem
Villamosmérnöki és Informatikai Kar
Automatizálási és Alkalmazott Informatikai Tanszék

Hajdú Patrik Zsolt

**AI AGENT TÁMOGATOTT
SZOFTVERFEJLESZTÉSI MÓDSZEREK ÉS
AI AGENT INTEGRÁCIÓS MEGOLDÁSOK
MEGISMERÉSE**

KONZULENS

Tóth Tibor

BUDAPEST, 2026

Tartalomjegyzék

1. Bevezetés	3
2. Kutatási háttér és Technológiai Architektúra.....	5
2.1 A szoftverfejlesztési információkeresés paradigmaváltása	5
2.2 Promptolási stratégiáim a gyakorlatban.....	5
2.3 Reaktív modellektől a Proaktív Ágensekig	6
2.4 Futtatási környezeteim és az AI Harness	6
2.5 Technológiai Architektúra	7
2.5.1 Orchestráció és Hosting	7
2.5.2 Backend, Adatbázis és Autentikáció	7
2.5.3 Frontend és Felhasználói Felület	8
3. Az ARSM Referenciaalkalmazás	10
3.1 A rendszer funkcióinak bemutatása	10
3.2 AI támogatott fejlesztés a gyakorlatban.....	10
3.2.1 Első Kihívás: Jogosultságkezelési mátrix és Állapotgép kialakítása.....	11
3.2.2 Második Kihívás: Mobil reszponzivitás és UI/UX finomhangolás	12
4. Validáció és Tesztelés.....	13
4.1 Fejlesztés közbeni validáció és biztonság.....	13
4.2 API és Adatbázis szintű tesztelés.....	13
4.2.1 HTTP Végpont (Endpoint) Tesztelés:	14
4.2.2 SQL Adatbázis (Database) Tesztelés:.....	14
4.3 Folyamatos Integráció.....	14
5. Összegzés és Szakdolgozati Jövőkép	16
5.1 A félév értékelése.....	16
5.2 Szakdolgozati jövőkép és technológiai célkitűzések	16
5.2.1 Éles üzembe állítás és funkcióbővítés	16
5.2.2 A tesztelési hierarchia kiterjesztése	17
5.2.3 Az ágens-alapú munkafolyamatok optimalizálása	17
5.2.4 Lokális modellek kipróbálása és tesztelése	17
6. Irodalomjegyzék.....	18
Függelék.....	19
Nyilatkozat generatív mesterséges intelligencia alkalmazásáról.....	19

1. Bevezetés

Informatikusként napjainkban azt tapasztalhatjuk, hogy a szakmai hírforrások jelentős része a mesterséges intelligencia (MI) térnyerésével, illetve annak a munkaerőpiacra gyakorolt hatásaival foglalkozik. Őszintén megvallva, bennem is megfogalmazódtak ezzel kapcsolatos aggodalmak. Mivel napi szinten követem az informatikai újdonságokat, folyamatosan szembesülök az ehhez kapcsolódó fejleményekkel, amelyek az elmúlt években a technológiai hírek túlnyomó többségét tették ki. Rendszeresen találkozom olyan bejelentésekkel, amelyek a piacon megjelenő legújabb, minden korábbinál fejlettebb modelleket mutatják be. Kiváló példa erre, amikor április 16-án megjelent a Claude Opus 4.7 [1] majd alig egy héttel később, április 23-án kiadták a ChatGPT 5.5-ös modellt [2] ismét átírva a technológiai erőviszonyokat. Ezt a robbanásszerű fejlődést látva érthető, hogy sokakban mind informatikusokban mind az informatika területen nem jártas emberekben bizalmatlanság vagy akár ellenállás alakult ki a technológiával szemben.

Gyermekkoromat és felnőtté válásomat végigkísérte a digitális technológia robbanásszerű fejlődése: az érintőképernyős okostelefonok, az okosórák vagy éppen a vezeték nélküli fülhallgatók mind az én generációm szeme láttára váltak a mindennapok elengedhetetlen eszközeivé. Mivel ezeket az innovációkat első kézből, szinte a megjelenésük pillanatától kezdve meg tapasztalhattam, rendkívül gyorsan a magabiztos és tudatos felhasználójukká váltam. Pontosan ezt a fajta korai, tapasztalati úton megszerzett rutint és természetességet érzem most a mesterséges intelligencia eszközeinek mindennapi használata során is.

Az Önálló Laboratórium keretében éppen ezért az „AI Agent támogatott szoftverfejlesztési módszerek és AI Agent integrációs megoldások megismerése” című témát választottam. Döntésemben motivált, hogy bár a mesterséges intelligenciát napi szinten alkalmaztam, és egyetemi tanulmányaim (például a Mesterséges Intelligencia, a kapcsolódó specializációs ágazati labor, valamint a Természetes nyelvi és szemantikus technológiák tantárgyak) során elsajátítottam a modellek architektúrális működését és elméleti hátterét, a gyakorlati szoftverfejlesztésben sokáig mégis a csevegőfelületekről történő, hagyományos vágólapos másolásra (copy-paste) hagytam. Ez a módszer nem volt kellően megbízható és messze állt a determinisztikus működéstől. Ez a

megközelítés gyakran vezetett hibákhoz és figyelmeztetésekhez a fejlesztőkörnyezetem beépített ellenőrző eszközei, valamint a statikus kódelemzők (például a SonarQube vagy a különböző szintaktikai ellenőrzők) részéről.

A választott témám célkitűzése így az volt, hogy felkutassam és megismerjem azokat a modern, automatizált munkafolyamatokat (agentic workflows), amelyekkel a mesterséges intelligencia segítségével megbízhatóbb, újrahasznosítható, jól tesztelhető és a determinisztikus kódgeneráláshoz a lehető legközelebb álló megoldásokat tudjak előállítani. Fontos tisztázni, hogy a dolgozatban szereplő „AI Agent” kifejezés alatt már nem az egyszerű, kérdés-felelet (chat) alapú nyelvi modelleket értem. Sokkal inkább olyan autonóm, a teljes projektkontextust átlátó intelligens asszisztensekről van szó, amelyek képesek a fájlrendszerben dolgozni, külső fejlesztői eszközöket alkalmazni, weboldallal interakcióba lépni és iteratív módon végrehajtani a rájuk bízott mérnöki részfeladatokat.

A kutatásomat nem kizárólag elméleti síkon végeztem el. A vizsgált módszertanok validálására egy valós üzleti logikát megvalósító referenciaalkalmazást is aktívan fejleszték. Ez a ARSM azaz „Appointment and Resource Scheduling Management”, ez egy autószerelő műhelyek részére készített időpontfoglaló és ügyfélnyilvántartó webes szoftver.

A dolgozatom a felvázolt célkitűzések mentén haladva mutatja be az elvégzett munkámat. A 2. fejezet az elméleti háttérrel és az alkalmazott technológiákat ismerteti. A 3. fejezet a referenciaalkalmazás implementációs lépéseit és az ágensek orchestrációját részletezi. A negyedik fejezet a tesztelési és validációs eljárásokat foglalja össze, végül az 5. fejezet az elért eredményeket értékeli és felvázolja a szakdolgozati továbbfejlesztés jövőképét.

2. Kutatási háttér és Technológiai Architektúra

Ebben a fejezetben azokat az elméleti és gyakorlati technológiai alapokat mutatom be, amelyekre a félév során elvégzett fejlesztői munkámat építettem. A kutatásom során egy modern, felhőalapú (cloud-native) alkalmazás elkészítése volt a cél, ahol a szoftverfejlesztési ciklus minden szakaszát autonóm mesterséges intelligencia ágensekkel támogattam meg.

2.1 A szoftverfejlesztési információkeresés paradigmaváltása

Szoftverfejlesztőként a munkaidőm jelentős részét a hibakeresés (debugging), a dokumentációk olvasása és a megfelelő információk felkutatása teszi ki. A kutatásom során azt tapasztaltam, hogy a statikus dokumentumanalízisen alapuló hagyományos keresés (a jól ismert „Ctrl + F” mechanizmus) egyre kevésbé hatékony a modern, komplex rendszerekben. Ha egy specifikáció „kliensként” hivatkozik egy entitásra, én pedig „felhasználóként” keresem, a hagyományos kereséssel rendre kudarcot vallottam.

A nagy nyelvi modellekkel (LLM) végzett munkám során ezt a folyamatot a szemantikus kontextus-értelmezésnek nevezzük. Amikor például hibaüzeneteket adtam be modellnek, azt figyeltem meg, hogy az MI az ok-okozati összefüggéseket is képes volt megérteni. Így egyből a saját projektem környezetére szabott megoldást kaptam, amivel rengeteg időt spóroltam meg, hiszen nem kellett [Stack Overflow](#), [Geeks for Geeks](#) vagy [Reddit](#)-en talált az én problémámra megoldást nyújtó kódok projektem környezetére szabásával foglalkoznom.

2.2 Promptolási stratégiáim a gyakorlatban

Munkám során egyértelművé vált, hogy a generált kód minősége alapvetően az utasításadás (promptolás) technikáján múlik. A fejlesztés során három fő stratégiát vizsgáltam és alkalmaztam:

- **Zero-shot prompting:** Előzetes kontextus nélküli azonnali kérdésfeltevéseket leginkább elméleti fogalmak tisztázására használtam. Kódolási feladatoknál azonban ez az a módszer, amivel gyakran tapasztaltam, hogy az MI „hallucinál” (ez annyit jelent, hogy nem helyes dolgokra magabiztosan mondja a modell, hogy helyes), mivel a modell nem ismerte a projektem specifikus architektúráját.

- **Few-shot prompting:** Ezt egy hatékony aranyközépútnak találtam. Ilyenkor a feladat pontos leírását 3-5 konkrét példával, valamint validációs szabályokkal támasztottam alá. Legnagyobb hátránya, hogy a magas kontextusablakot-igényel, amely hosszú távon a modell "emlékezetkieséséhez" vezetett nálam ez pedig hallucinációt okozott.
- **Chain of Thought (CoT):** A fejlesztés során ezt a megközelítést alkalmaztam a leggyakrabban. Ennek lényege a közös gondolkodás volt: a végleges kód egy prompt-ból való előállítása helyett iteratív párbeszédet folytattam az MI-vel az architektúráis döntéseimről. Ezzel a módszerrel drasztikusan csökkentettem a hibák számát, és a projekt során meghozott mérnöki döntéseket így mind én tettem meg!

2.3 Reaktív modellektől a Proaktív Ágensekig

A hagyományos LLM interakciókat alapvetően reaktívnak éltem meg: kérdeztem, a modell pedig válaszolt. A szemléletváltást a munkámban a proaktív, autonóm AI Ágensek integrálásával értem el. Ezeknél az ágenseknél az alapvető tudásukon túl felruháztam őket a cselekvéshez szükséges eszközökkel.

Az interakciós módok skáláján a legalapvetőbbként a Tervezői (Plan) módot alkalmaztam. Majd ezt követően az Ágens (Agent) módot alkalmaztam, ahol az MI a teljes projektet át látva képes volt fájlok létrehozására, kód futtatásra, terminálpárancsok futtatására és a többi... Így egyszerű programozóból tényleges rendszer-orkesztrátorrá léptem elő, hirtelen én lettem a Senior fejlesztő, aki irányított és tervezett.

2.4 Futtatási környezeteim és az AI Harness

Mivel egy felhőben futó LLM önmagában képtelen változtatni és használni az én lokális gépemén elérhető erőforrásokat, egy keretrendszert (AI Harness) kellett alkalmaznom, amely összekötötte a modell "agyát" az én fejlesztői környezetemmel.

A projekt során a GitHub Copilot és a Claude Code kettősére támaszkodtam. Emellett a keretrendszereket projekt-specifikus szabályokkal (.github/copilot-instructions.md és a projekt gyökerében lévő CLAUDE.md fájlokkal) láttam el.

2.5 Technológiai Architektúra

Az ágensek képességeinek validálására egy autószerelő műhelyek számára készült időpontfoglaló rendszert terveztem majd valósítok jelenleg is meg. Az architektúra kialakításakor a Clean Architecture elveit követtem, élesen szétválasztva a backend, a frontend és az adatbázis rétegeit.

2.5.1 Orchestráció és Hosting

A projektem technológiai gerincének a C# programozási nyelvet és a .NET 10 keretrendszert választottam. Az elosztott szolgáltatások, mint az `AutoService.AppHost`, `AutoService.WebUI`, `AutoService.ApiService` és az `AutoService.ServiceDefaults` modulok összehangolására konzulensem ajánlására .NET ökoszisztémában egyre meghatározóbbá váló Aspire orkesztrációs keretrendszert alkalmaztam [3]. A fejlesztés során ez a réteg felelt a lokális szolgáltatásaim és az adathozzáférés felügyeletéért. Az adatperzisztenciát egy dedikált, állandó tárterülettel (volume) ellátott PostgreSQL Docker konténerben valósítottam meg. Az Aspire eszköztárának segítségével a hálózati kapcsolatokat és a kapcsolati karakterláncokat (connection strings) dinamikusan injektáltam a futtatókörnyezetekbe, amivel sikeresen kiküszöböltem a statikus konfigurációs fájlok (például a kézzel rögzített portok vagy elavult jelszavak) használatából eredő tipikus fejlesztői hibákat.

2.5.2 Backend, Adatbázis és Autentikáció

A backend szolgáltatásaim magját egy .NET 10 alapokon nyugvó REST API adja. Az adatperzisztenciát Entity Framework Core (EF Core) segítségével valósítottam meg egy relációs PostgreSQL adatbázis felett. A domain modell tervezése során a felhasználói entitások kezelésére a Table-Per-Hierarchy (TPH) öröklődési mintát alkalmaztam. Ennek köszönhetően a személyeket egyetlen közös adatbázistáblában tárolom, ahol egy jogosultságot egyértelműsítő oszlop alapján különítem el a rendszerbe bejelentkezni jogosult szerelő (Mechanic) és a pusztán passzív adatként kezelt ügyfél (Customer) rekordokat.

Id	FirstName	MiddleName	LastName	Email	PhoneNumber	PersonType	Specialization	Expertise	IdentityUserId	ProfilePicture	ProfilePictureCa...
1	Anna	Maria	Tóth	anna.toth@example.com	+36304445566	Customer					
2	Benice		Farkas	benice.farkas@example.com	+36305556677	Customer					
3	Csilla	Kata	Varga	csilla.varga@example.com		Customer					
4	David		Kiss	david.kiss@example.com	+36306667788	Customer					
5	Emese		Lékatos	emese.lekatos@example.com		Customer					
6	Gaber		Kovacs	gabriel.kovacs@example.com	+36301112233	Mechanic	GasolineAndDiesel	Engine, Transmission, B...	ef1ed919-4632-4029-...		
7	Mate		Szabo	mate.szabo@example.com	+36303344555	Mechanic	All	Engine, Transmission, B...	59e46237-9534-4629-...		
8	Duplicate			matix.phone.user.17787850...	+36309998888	Mechanic	GasolineAndDiesel	Engine, Brakes	79f2e4bc-1411-4345-...		
9	Duplicate		Phone	duplicate.phone.177878504...	+36307770991	Mechanic	GasolineAndDiesel	Engine, Brakes	7cd8d8bc-c2b1-4306-...		
10	Patrik	Zsolt	Hajdú	hagdu.patrik1023@gmail.com	+36205602016	Mechanic	All	Engine, Transmission, F...	769f782e-0949-43e8-...	[[{"key": "ProfilePicture", "value": "image.png"}]]	
11	Peter		Hagyó	mechanic-owner-12@custo...		Customer					
12	Patrik	Zsolt	Hajdú	mechanic-owner-16@custo...		Customer					

„1. ábra: Az adatbázis séma bemutatása (teszt adatokkal)”

A rendszer hitelesítési (Authentication) rétegének garantálására egy, állapotmentes JWT (JSON Web Token) alapú megoldást implementáltam. A biztonsági kockázatok (például XSS támadások) minimalizálása érdekében a tokeneket kizárólag a szerver által beállított, HttpOnly, Secure és SameSite=Strict attribútumokkal rendelkező sütitben (autoservice_at, autoservice_rt) tárolom, így a kliensoldali futtatókörnyezet egyáltalán nem férhet hozzájuk. A kijelentkezési folyamat biztonsága érdekében a rendszer, azonnal és visszavonhatatlanul érvényteleníti a kompromittálódott vagy lejárt hozzáférési tokeneket.

A végpontok védelmét és a hozzáférés-szabályozást (Authorization) szerepkör-alapú megközelítéssel, az ASP.NET Identity rendszer integrálásával valósítottam meg. Az API végpontjaimat szigorú házirendekkel (Policies) védem: például a rendszerbe történő új szerelői regisztráció kizárólag az Admin szerepkörrel rendelkező felhasználók számára engedélyezett, míg az időpontok állapotának módosítását csak az adott munkára hozzárendelt szerelők végezhetik el.

A rendszer stabilitása és a túlterheléses (Brute-force) támadások elleni védelem érdekében az architektúrát egy forgalomkorlátozó (Rate Limiting) réteggel is rendelkezik. A hitelesítési végpontokon szigorú szabályok vannak: a bejelentkezési kísérleteket percenként tíz, míg a token-frissítéseket percenként húsz kérdésben maximáltam IP-címenként. Ezt a védelmet egy egyedi köztesréteggel (Middleware) biztosítom, amely a túl gyakori bejelentkezési próbálkozások esetén egy automatikus, háromperces IP-alapú tiltást (Ban) léptet életbe, ezzel is védve az adatbázist a rosszindulatú forgalomtól.

2.5.3 Frontend és Felhasználói Felület

A kliensoldali alkalmazásomat React 19 és Vite alapokon építettem fel. A formázáshoz TailwindCSS-t, a weboldal állapot kezeléshez a Zustand könyvtárat használtam. A kliensoldali jogosultság-ellenőrzést (Protected Routes) úgy alakítottam ki,

hogy az a szerepkörök alapján szigorúan szabályozza a nézetekhez való hozzáférést. Az UI tervezése és a reszponzivitás kialakítása során a Multi-Agent megközelítést (dedikált Frontend és UI/UX Stylist ágensek egyidejű alkalmazását) követtem, amellyel garantálni tudtam a kódom minőségét, a „Low Design” elveit és a design-rendszerem konzisztenciáját

3. Az ARSM Referenciaalkalmazás

A kutatásom gyakorlati megvalósításaként és a mesterséges intelligencia ágensek képességeinek validálására egy Proof of Concept (PoC) alkalmazást fejleszték. Az ARSM (Appointment and Resource Scheduling Management) egy autószerelő műhelyek számára készült időpontfoglaló és ügyfélnyilvántartó webes szoftver. A szoftver nem csupán elméleti koncepció, hanem valós piaci igényekre válaszol: a projekt célja, hogy a jövőben ténylegesen kiváltsa a kis- és középvállalkozó autószerelő műhelyek (például a saját családi környezetemben is tapasztalt) táblázatkezelőkre (pl. Google Sheets) épülő, nehezen skálázható és karbantartható adminisztrációs folyamatait. Ebben a fejezetben az alkalmazás főbb funkcióit, valamint az ágensek segítségével megoldott konkrét mérnöki kihívásokat mutatom be.

3.1 A rendszer funkcióinak bemutatása

Az alkalmazás célja a műhelyek napi adminisztrációs és ütemezési feladatainak digitalizálása. A fejlesztés során az alábbi főbb modulokat valósítottam meg:

- **Hitelesítés és jogosultságkezelés:** A rendszer zártkörű, csak regisztrált munkatársak (Szerelők és Adminisztrátorok) férhetnek hozzá.
- **Naptár és Időpontkezelő (Scheduler):** A szoftver központi eleme egy interaktív naptárnézet, amelyben a szerelők napi és havi bontásban tekinthetik át a javításra váró gépjárműveket. Az időpontok esetében elkülönítettem a „Beérkezés ideje” és a „Határidő” fogalmát.
- **Ügyfél és Jármű Registry:** Dedikált felületet alakítottam ki az ügyfelek adatainak, valamint a hozzájuk tartozó gépjárművek specifikációinak (például évjárat, futásteljesítmény, motorteljesítmény) nyilvántartására.

3.2 AI támogatott fejlesztés a gyakorlatban

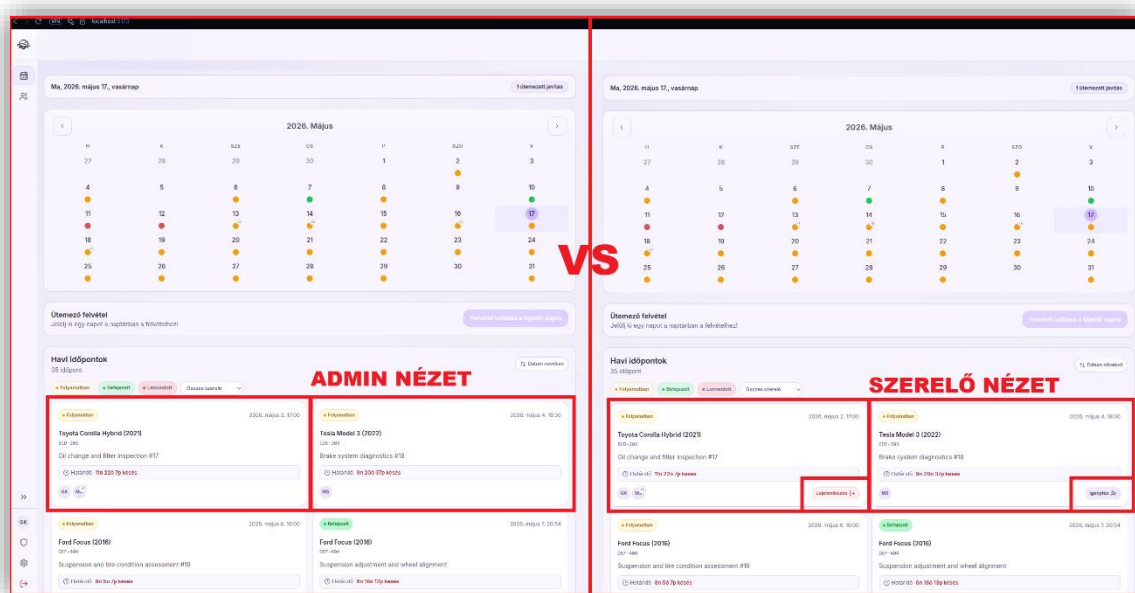
fejlesztés során az AI ágensek (kifejezetten a Claude Code és a GitHub Copilot) használata nem csupán a kódgenerálást gyorsította fel, hanem az architektúrális problémák megoldásában is kulcsszerepet játszott. Két kiemelt esettanulmányon keresztül mutatom be, hogyan orchestráltam az ágenseket a komplex kihívások leküzdéséhez.

3.2.1 Első Kihívás: Jogosultságkezelési mátrix és Állapotgép kialakítása

Az időpontok kiosztása során egy kritikus logikai problémába ütköztem: hogyan kerülhető el, hogy egy felvett szerelési munka gazdátlan maradjon, illetve hogyan biztosítható, hogy a felhasználók csak a saját szerepkörüknek megfelelő gombokat lássák?

Ennek megoldására egy „Zero-Orphan” (gazdátlan munka tilalma) szabályt definiáltam. Az Orchestrator ágens segítségével egy szigorú állapotgépet (State Machine) terveztettem és implementáltam a kódba. A logika a következőképpen állt össze:

- Ha a bejelentkezett felhasználó Adminisztrátor, számára elrejtettem az „Igénylés” és „Lejelentkezés” gombokat, ő kizárólag egy legördülő menüből oszthatja ki a munkát.
- Ha a felhasználó egy Szerelő, aki egyedülként van beosztva egy munkára, a felületről eltűnik számára a „Lejelentkezés” opció, megakadályozva ezzel a munka elhagyását. A gomb csak akkor jelenik meg újra, ha legalább egy másik kolléga is csatlakozik a feladathoz.

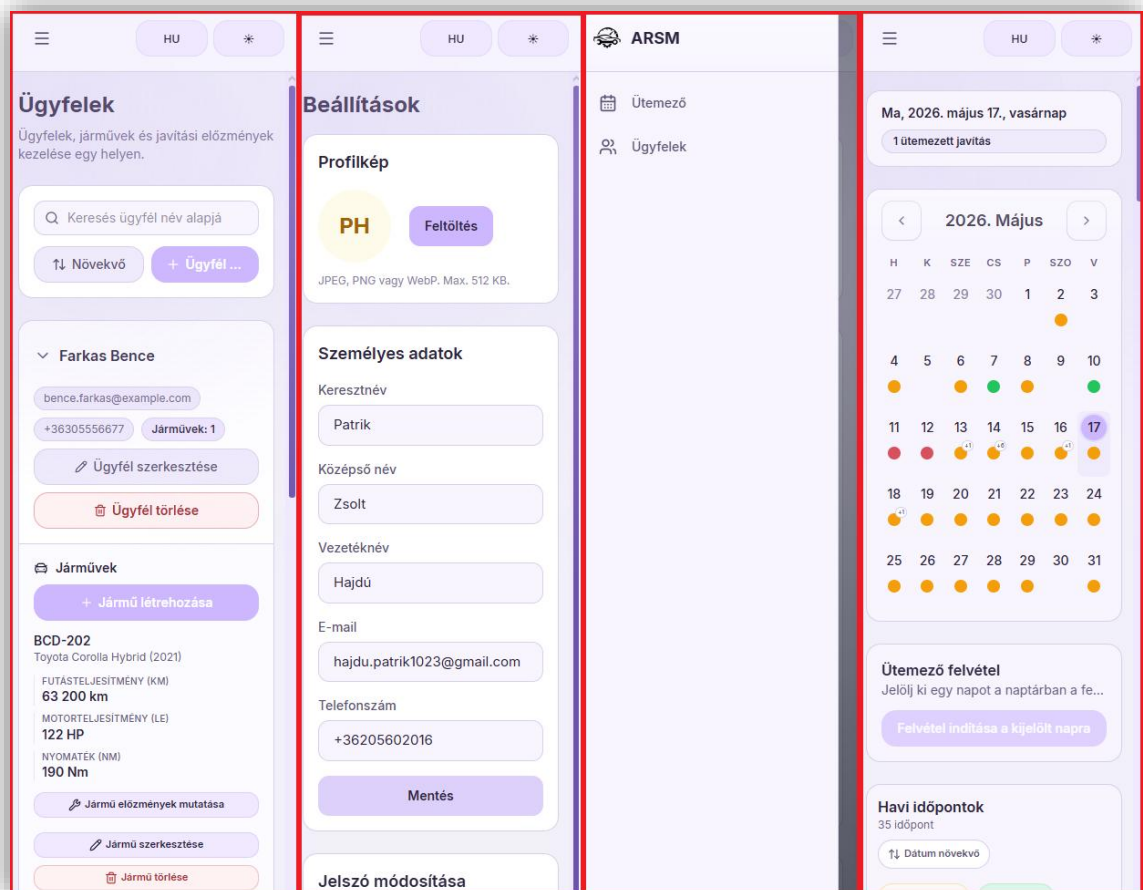


„2. ábra: Az ARSM naptárnézete és a szerepkör-alapú akciógombok.”

3.2.2 Második Kihívás: Mobil reszponzivitás és UI/UX finomhangolás

A fejlesztés előrehaladtával a felhasználói felület asztali környezetben jól nézett már ki, azonban a kisebb, 320 pixel széles mobilnézeteken a konténerekből hiányzó túlsordulás-védelem miatt az elemek egymásba csúsztak.

A probléma orvoslására egy dedikált UI/UX Stylist ágens-szabályrendszer (egy `.github/agents/` mappában elhelyezett instrukciós fájl) hoztam létre. Ennek keretében szigorúan megköveteltem a Frontend ágenstől a „Kevesebb több” (Less is more) tervezési elv alkalmazását és a modern reszponzív irányelvek betartását [4]. Az ágens a működése során végigpásztázta a kódbázist, és felszámolta a túlzottan egymásba ágyazott (nested-box) elrendezéseket. A flex konténerekhez automatikusan hozzáadta a megfelelő `min-w-0` és `truncate` Tailwind osztályokat, biztosítva a szövegek helyes csonkolását és a reszponzív megjelenést.



„3. ábra: A 320px-es mobil reszponzivitás bemutatása.”

4. Validáció és Tesztelés

Egy modern szoftverfejlesztési projekt elengedhetetlen része a folyamatos és automatizált minőségbiztosítás. A kutatásom során kiemelt figyelmet fordítottam arra, hogy a mesterséges intelligencia ágensek ne csupán a kód előállításában, hanem annak validálásában és tesztelésében is aktívan részt vegyenek. A szigorúan szabályozott ágens-architektúrával (Harness) és a dedikált készségekkel (skills) értem el azt, hogy az MI-alapú kódgenerálás véletlenszerűsége (hallucinációja) egy szinte teljesen determinisztikus, megismételhető és megbízható mérnöki folyamattá váljon.

4.1 Fejlesztés közbeni validáció és biztonság

A szoftverminőség garantálása érdekében a tesztelést már a kódolási fázisban megkezdtem. Az ágens-alapú munkafolyamatomban (Agentic Workflow) dedikált minőségbiztosítási kapukat (Quality Gates) iktattam be. Miután a Backend vagy Frontend ágensek befejeztek egy feladatot, egy specializált, gyors modellre épülő validate ágens veszi át az irányítást.

Ennek a perszónának az volt a feladata, hogy minden módosítás után autonóm módon lefuttassa a backend .NET build folyamatait és a frontend TypeScript típusellenőrzését (tsc). Az ágens nemcsak a szintaktikai helyességet vizsgál, hanem automatikusan kikényszeríti a SOLID elvek betartását, a fájlméret-korlátokat (például 500 sornál hosszabb forrásfájlok darabolását), valamint azonnali biztonsági auditot és javítást (npm audit fix és dotnet list package --vulnerable) is végrehajt minden fejlesztési iterációban. Ezzel biztosítottam, hogy kizárólag biztonságos és lefordítható kód készüljön.

4.2 API és Adatbázis szintű tesztelés

Az alkalmazás üzleti logikájának és adatintegritásának validálását jelenleg két stabil, automatizált pillérre: a HTTP végpontok és az SQL adatbázis-séma tesztelésére építettem. Ennek karbantartására külön ágenseket definiáltam, amelyek csak szigorú feltételek (Trigger Gates) teljesülése esetén futnak le, garantálva a determinisztikus működést.

4.2.1 HTTP Végpont (Endpoint) Tesztelés:

Létrehoztam egy http-endpoint-test ágenszt, amely az API szerződések (contracts) változása esetén automatikusan frissíti a tests/API//*.http fájlokat. Ez az ágens önállóan generálta a pozitív (Happy-path) és negatív teszteseteket. A szkriptekkel nemcsak az adatok sikeres lekérdezését vizsgáltam, hanem szigorúan teszteltem a szerepkör-alapú hozzáféréseket (401 Unauthorized, 403 Forbidden) és a bemeneti validációs hibákat (400 Bad Request, 422 Unprocessable Entity) is. A tesztek környezeti változókat (Environment Variables) használnak, így a titkosított adatok sosem kerültek beégetésre (hardcoding) a forráskódba.

4.2.2 SQL Adatbázis (Database) Tesztelés:

Az adatbázis szintű konzisztencia ellenőrzését a sql-database-test ágensre bízam. Ez az ágens kizárólag olvasható (SELECT) lekérdezéseket generált a tests/Database//*.sql mappába, teljesen elkerülve a destruktív (DML/DDL) műveleteket. Ezen szkriptekkel validálom az Entity Framework által generált sémák helyességét, az indexek meglétét, valamint az összetett üzleti logikákat.

4.3 Folyamatos Integráció

A lokális validáción túl a projektet egy GitHub Actions alapú, folyamatos integrációs (CI/CD) munkafolyamattal (dotnet.yml) is kiegészítettem. A jelenlegi fázisban a csővezeték (pipeline) úgy konfiguráltam, hogy a főágot (main branch) érintő minden közvetlen módosításnál (push) és beolvasztási kérelemnél (pull request) automatikusan lefusson, fókuszba helyezve a stabilitást és a szoftverbiztonságot.

A maximális megbízhatóság elérése érdekében egy mátrix-alapú végrehajtási stratégiát (Matrix Strategy) vezettem be. Ez azt jelenti, hogy a backend és a frontend ellenőrzési folyamatokat párhuzamosan, három különböző operációs rendszeren (ubuntu-latest, windows-latest és macos-latest) futtatom és validálom, így még a főágra kerülés előtt kiszűröm az esetleges platformfüggő anomáliákat. A munkafolyamat aszinkron módon, két különálló feladatban (job) fut le:

- A **backend** ágon a .NET 10 környezet felépítése után elvégzem a függőségek helyreállítását, a kód fordítását és a rendelkezésre álló tesztek futtatását.

- A **frontend** ágon Node.js 22 környezetben (a gyorsítótárazott függőségek telepítése után) végrehajtom a szigorú TypeScript típusellenőrzést (tsc --noEmit), garantálva a React komponensek típusbiztonságát.

A CI/CD architektúra kialakításakor kiemelten támaszkodtam a DevSecOps ágens iránymutatásaira, aminek köszönhetően rendkívül hatékony biztonsági kapukat építettem az automatizációba. A backend ágon egy bash szkript segítségével kikényszerítettem a NuGet csomagok tranzitív sérülékenységvizsgálatát (dotnet list package --vulnerable), a frontend ágon pedig az NPM csomagok auditálását magas riasztási szinten (npm audit --audit-level=high). A munkafolyamatot úgy állítottam be, hogy amennyiben bármelyik ág ismert sebezhetőséget talál a használt külső csomagokban, a fordítási folyamat azonnal elbukik, megakadályozva ezzel a kompromittálódott kód élesítését.

5. Összegzés és Szakdolgozati Jövőkép

Az Önálló Laboratórium keretében végzett kutatásom és az ARSM referenciaalkalmazás gyakorlati fejlesztése során sikeresen validáltam a többágenses (Multi-Agent) szoftverfejlesztési módszertanok létjogosultságát. Ebben a fejezetben röviden összegezem a félév során elért mérnöki eredményeket, és felvázolom azokat a technológiai és kutatási mérföldköveket, amelyeket a szakdolgozati félévemben tervezek megvalósítani.

5.1 A félév értékelése

A félév legnagyobb eredményének egy olyan zárt, hierarchikus ágens és készségrendszer (Harness) kialakítását tekintem, amely a hagyományos, passzív kódgenerálást egy proaktív, mérnökiileg szigorúan ellenőrzött folyamattá alakította. A dedikált perszónák (Orchestrator, Backend, Frontend, Validate) alkalmazásával és a folyamatos, CI/CD szintű automatizált HTTP és SQL teszteléssel elértem a determinisztikushoz közeli kódminőséget. A fejlesztés sebessége jelentősen megnőtt, így a figyelmemet a repetitív feladatok helyett a magas szintű üzleti logika és a rendszer-orkesztráció felé tudtam fordítani.

5.2 Szakdolgozati jövőkép és technológiai célkitűzések

Az Önálló Laboratóriumban lerakott alapokra építve a szakdolgozatom keretében a rendszert egy működő terméké, a kutatásomat pedig egy még átfogóbb technológiai elemzéssé kívánom bővíteni. A következő félévre az alábbi főbb célkitűzéseket fogalmaztam meg:

5.2.1 Éles üzembe állítás és funkcióbővítés

A szoftverfejlesztési folyamat végső célja egy stabilan működő, telepített alkalmazás átadása. A szakdolgozat végére az ARSM rendszert éles szerverkörnyezetben fogom publikálni, hogy a megcélzott autószerelő műhely a mindennapokban is használhassa, véglegesen leváltva a táblázatkezelős megoldásokat. Az alkalmazást emellett az éles tesztüzem tapasztalatai és a valós felhasználói igények alapján új funkciókkal is bővíteni szeretném, ha sikerül elérnem elég felhasználót.

5.2.2 A tesztelési hierarchia kiterjesztése

A minőségbiztosítást még szigorúbb, több szinten történő, szabályalapú teszteléssel fogom megerősíteni. Ennek legfontosabb lépése a Playwright keretrendszerre épülő E2E (End-to-End) tesztelés teljes körű bevezetése, amellyel a felhasználói felület interakcióit és a teljes kliensoldali életciklust automatizáltan fogom majd vizsgálni, kiegészítve a már meglévő API és adatbázis teszteket.

5.2.3 Az ágens-alapú munkafolyamatok optimalizálása

Az AI orkesztráció hatékonyságának további növelése érdekében kiemelt fókuszot helyezek a kontextusablak méretének optimalizálására.

5.2.4 Lokális modellek kipróbálása és tesztelése

Mivel az MI iparága rendkívül gyorsan fejlődik, a kutatásom szerves része marad, a piacon új addig megjelenő eszközök és modellek folyamatos nyomon követése és integrálása. Ezen felül vizsgálni fogom a felhőfüggetlen, lokális nagy nyelvi modellek (Local LLMs) alkalmazhatóságát a szoftverfejlesztésben. Ehhez a munkaállomásom hardveres specifikációi Intel Core i9-14900K processzor, 32 GB DDR5 5800MHz memória és egy 16 GB VRAM-mal rendelkező RTX 4090 Laptop GPU ez pedig egy szokványos laptopnál nagyobb számítási kapacitás átlagosan így nagyobb teret ad nekem a kísérletezéshez.

6. Irodalomjegyzék

- [1] Anthropic, "anthropic.com," Anthropic, 16. április 2026. [Online]. Available: <https://www.anthropic.com/claude/opus>.
- [2] OpenAI, „openai.com,” OpenAI, 23. április 2026. [Online]. Available: <https://openai.com/hu-HU/index/introducing-gpt-5-5/>.
- [3] Aspire, „aspire.dev,” Microsoft, 22. április 2026. [Online]. Available: <https://aspire.dev/get-started/what-is-aspire/>.
- [4] TailwindCss, „tailwindcss.com,” Tailwind Labs Inc., 16 március 2026. [Online]. Available: <https://tailwindcss.com/docs>.

Függelék

Nyilatkozat generatív mesterséges intelligencia alkalmazásáról

- ☐ **Nem használtam** semmilyen generatív MI segédeszközt.
- ☒ **Használtam** generatív MI segédeszközt. Az MI-vel generált tartalmakat ellenőriztem, a generált kimenetek valóságtartalmáról meggyőződtem, az alábbi táblázatban megfelelően jelöltem minden használatot.

Felhasználási módok	Generatív MI eszköz(ök) neve	Érintett részek (fejezet, oldalszám, hivatkozás)	Használat becsült aránya (felhasználási módonként)
Irodalomkutatás	Google Gemini 3.1 Pro, Claude Sonnet 4.6	2. fejezet (Kutatási háttér és Technológiai Architektúra)	30%
Prompt lényegi része	„Listázd ki nekem a 2026-os best practice-eket a Multi-Agent LLM workflow-król és a modern szoftverfejlesztési AI eszközökről (pl. Cursor, Claude Code).” - Ehhez hasonló prompt-okat adtam meg!		
Programkód generálása	ChatGPT 5.5/5.3 Codex, Claude Opus 4.6/4.7	3. fejezet (A referenciaalkalmazás implementációja).	85-90%
Prompt lényegi része	„Hozd létre az AutoService alkalmazás frontendjén a z ütemező oldalon a naptár nézetet. Implementáld 320px-es mobil nézetig reszponzívan az alábbiakban kért elemet és kövesd a 2026-os legjobb ajánlásokat. Használd a már elkészített stílus profilokat az implementáció során.” - Ehhez hasonló prompt-okat adtam meg!		
Új ötletek, megoldási javaslatok generálása	ChatGPT 5.5/5.3 Codex, Claude Opus 4.6/4.7	3. fejezet (Architektúra), 4. fejezet (Validáció és Tesztelés)	50%
Prompt lényegi része	„Sorolj fel három hatékony megoldást hogyan kéne bővítenem a CI/CD pipeline-ómat, hogy a playwrightos E2E tesztelést be tudjam vezetni!” - Ehhez hasonló prompt-okat adtam meg!		
Vázlat létrehozása (szövegstruktúra, vázlatpontok)	Google Gemini 3.1 Pro	A dokumentum logikai felépítése.	20%
Prompt lényegi része	„Segíts elkészíteni egy 10-15 oldalas dokumentáció vázlatot a BME AUT Önlab sablon és a maximális pontozási követelmények alapján.” - Ehhez hasonló prompt-okat adtam meg!		
Szövegblokkok létrehozása	Google Gemini 3.1 Pro	Összes fejezetben a Word-ön túl nyelvhelyesség	40%

		vizsgálat és mondat struktúra ellenőrzésére.	
Prompt lényegi része	"Nézd meg át, hogy a helyesírás megfűlő a leírtaaknak: ... és követik-e szigorúan E/1-es követelményt." - Ehhez hasonló prompt-okat adtam meg!		
Képek generálása illusztrációs célból	Google Nano Banana 2.0 Pro	A projekt összes illusztrációjához	100%
Prompt lényegi része	„Készíts nekem 4K felbontású teljesen fehér háttérű logót, amelyen szerepel egy autó, villáskulcs és fogaskerék” - Ehhez hasonló prompt-okat adtam meg!		
Adatvizualizáció, grafikonok generálása adatpontok alapján	-	-	-
Prompt lényegi része	-		
Prezentáció készítése	Google Gemini 3.1 Pro	A prezentáció logikai felépítése. Design ötletek.	30%
Prompt lényegi része	„Segíts elkészíteni egy 10 diás vázlatot a BME AUT Önlab sablon és a maximális pontozási követelmények alapján.” - Ehhez hasonló prompt-okat adtam meg!		
Egyéb (nevezze meg)	-	-	-
Prompt lényegi része	-		
Összesített százalékos érték (a feladat érdemi részére nézve)			75%
Összesített érték rövid, szöveges indoklása: Mivel az Önálló Laboratórium témám kifejezetten az AI ágensek szoftverfejlesztésben betöltött szerepének vizsgálata és integrációja volt, az eszközöket (Codex, Claude Code, Gemini) a kódgenerálás és a szoftverarchitektúra megtervezése során elsődleges eszközként, magas arányban. A dokumentációírás során az MI a gondolatmenet strukturálásában nyújtott segítséget, de a végső tartalmat, a mérnöki döntések (pl. UI/UX szabályok, CI/CD pipeline) megfogalmazását és a validációt én magam végeztem és ellenőriztem minden esetben (automatikus elfogadás joga soha nem volt az ágenseknek).			